

SNOWFLAKE – LECTURE NOTES

Snowflake is a cloud-native OLAP DBMS built from scratch to exploit disaggregated storage and elastic compute in public clouds, while exposing a “no-tuning” SQL warehouse as a managed service.

1. Historical context and motivation

- In the 2000s many specialized OLAP engines appeared (Vertica, Greenplum, MonetDB, Vectorwise, ParAccel), and in the early 2010s many organizations tried to run SQL on top of Hadoop/HDFS with systems like Hive, Presto, Impala, and Stinger, but these were self-managed, on-prem systems that were hard to operate and tune.^{[2][1]}
- Google’s Dremel paper (2011) and the subsequent BigQuery service demonstrated interactive SQL over nested data at large scale, while Amazon licensed ParAccel and launched Redshift (2013) as a managed shared-nothing warehouse in the cloud.^{[1][2]}
- Snowflake was founded in 2012 by engineers with Oracle and Vectorwise backgrounds to build a **cloud-first** data warehouse as a pure SaaS product with elastic compute, shared cloud storage, and strong SQL semantics, not based on Hadoop or PostgreSQL.^{[2][1]}

2. High-level design goals

- Provide an enterprise-ready, transactional, SQL data warehouse “as a service”: no hardware provisioning, no physical design, no index tuning, and minimal operational knobs; users just load data and run queries.^[2]
- Exploit cloud properties that classic shared-nothing systems struggle with:
 - Independently scalable compute and storage.
 - Frequent membership changes (node failures, resizing).
 - Need for online upgrades and continuous availability.^[2]
- Support both classic relational workloads and modern semi-structured, schema-less data (JSON, Avro, XML) at near-relational performance through columnar storage, type inference, and pruning.^{[1][2]}

3. Multi-cluster, shared-data architecture

Snowflake introduces a **multi-cluster, shared-data** architecture that separates storage, compute, and control into three logical layers:^{[1][2]}

3.1 Data storage layer

- Table data and large query results are stored in cloud object stores (e.g., Amazon S3), which provide very high durability and availability but higher latency and CPU overhead per I/O compared to local disks.^{[1][2]}
- Tables are horizontally partitioned into large, immutable files in a hybrid columnar (PAX) format: each file stores compressed column segments plus a header containing offsets, enabling range GETs to fetch only required columns.^[2]
- The same storage is used for:
 - Base table files (micro-partitions).
 - Temporary spill files when operators (join, sort, group-by) exceed memory.
 - Materialized query results, eliminating the need for server-side cursors.^{[1][2]}

3.2 Virtual warehouses (compute layer)

- A **Virtual Warehouse (VW)** is an elastic cluster of worker nodes (VMs) that executes queries; users see abstract T-shirt sizes (XS, S, ...) instead of specific node counts and types.^{[2][4]}
- Each query runs on exactly one VW; workers are not shared across warehouses, which provides strong performance isolation between workloads and organizational units.^[2]
- Creating, resizing, suspending, or resuming a VW affects only compute resources: tables remain in shared storage, so there is no data rebalancing during scaling.^[2]
- Each worker node maintains a **local disk cache** of file headers and column chunks that it has read from S3; caches are shared among all worker processes on the node and use a simple LRU policy.^{[1][2]}
- The optimizer assigns input files to workers using **consistent hashing** over file names so that the same file tends to map to the same worker, maximizing cache reuse and avoiding duplicated caching.^{[1][2]}

3.3 Cloud services layer

- The Cloud Services layer is the “brain” and is heavily multi-tenant and long-lived; it includes:
 - Query management and optimization.
 - Transaction management and concurrency control.
 - Security and authentication.

- Metadata and statistics storage (implemented using a transactional key-value store, e.g., Foundation DB in recent designs).^{[1][2]}
- Services are replicated across availability zones and accessed via RESTful interfaces behind load balancers, enabling high availability and transparent failover for control-plane operations.^{[2][1]}

4. Execution engine and compute elasticity

4.1 Execution model

- Snowflake implements its own **columnar, vectorized, push-based** execution engine: operators process batches of thousands of tuples in columnar format and push vectors downstream, avoiding Volcano-style pull overhead and unnecessary materialization.^{[1][2]}
- The engine uses pre-compiled C templates for operator kernels for common vector data types, while LLVM is used mainly for serialization/deserialization and networking, rather than aggressive per-query code generation.^[1]
- There is no buffer pool and no in-place table updates; operators spill to local disk or S3 when necessary, which allows executing arbitrarily large analytical queries while using memory primarily for computation instead of caching.^{[2][1]}

4.2 Caching, skew handling, and work stealing

- Local caches act as a **compute-side caching layer** on top of disaggregated storage, hiding much of S3's latency once caches warm up.^{[2][1]}
- **File stealing**: when a worker process finishes scanning its assigned files, it can request more work; peers “transfer ownership” of remaining files, but the requesting worker downloads data directly from S3 to avoid overloading stragglers.^{[1][2]}
- Cache contents are not proactively reshuffled when the VW membership changes; instead, consistent hashing plus LRU replacement gradually adapt cache state after failures or resizes, avoiding a distinct “degraded” mode.^[2]

4.3 Flexible compute (burst scaling within a query)

- For plan fragments that process large volumes of data (e.g., large joins or aggregations), Snowflake can temporarily deploy extra worker nodes as **flexible compute** to accelerate execution.^[1]

- Flexible compute workers behave as additional scan/compute nodes and write results back to storage as if materializing a table, enabling elastic scale-out for heavy sub-plans without permanently resizing the user's main VW.^[1]

5. Transaction model, pruning, and optimization

5.1 Snapshot isolation and MVCC

- Snowflake targets analytic workloads dominated by large reads, bulk loads, and batch updates and implements ACID transactions using **Snapshot Isolation (SI)** on top of MVCC.^[2]
- Because table files on S3 are immutable, updates are modeled as **file-set versioning**: each table version corresponds to a set of file identifiers stored in the metadata KV store; updates insert new files and mark old files as obsolete in metadata.^[2]
- Reads operate over a consistent snapshot defined by a transaction timestamp or identifier, and the engine sees a fixed set of immutable files for the duration of query execution, simplifying concurrency control in the execution layer.^[2]

5.2 Statistics and pruning instead of indexes

- Snowflake does not expose user-defined secondary indexes; instead it relies on **min/max-based pruning** ("zone maps", "data skipping") at the file/micro-partition level.^{[1][2]}
- For each file and column the system stores metadata such as min, max, row counts, null counts, and distinct counts; during optimization, predicates (including some expressions, e.g., date truncation) are evaluated against metadata to eliminate files that cannot satisfy the predicate.^{[1][2]}
- Simple statistics are kept in sync automatically for internal formats (Snowflake micro-partitions), and the optimizer uses them to estimate the number of micro-partitions per pipeline and drive both cost estimation and resource decisions.^[1]
- Dynamic pruning: during execution, join operators can collect build-side key distributions and push filters to probe-side scans, skipping entire files beyond what static pruning can achieve; Bloom-filter-based techniques complement this for some joins.^{[2][1]}

5.3 Adaptive optimization

- Snowflake's optimizer is Cascades-style, top-down, cost-based, but defers some decisions to runtime to reduce bad plan choices under uncertain statistics.^[2]

- An example is **adaptive aggregation placement**: the optimizer may insert additional aggregation operators below joins but only enable them at runtime when observed statistics indicate that early aggregation will be beneficial, effectively performing runtime plan specialization.^[1]

6. Data storage: micro-partitions and semi-structured data

6.1 Micro-partitioned columnar storage

- Internally, tables are broken into hundreds of thousands to millions of **micro-partitions**: immutable columnar files typically representing 50–500 MB of raw uncompressed data, often compressing down to ~16 MB per file.^{[3][1]}
- Snowflake continuously and transparently **re-clusters** micro-partitions in the background based on observed query patterns, progressively reorganizing data to improve pruning and scan locality without explicit user-managed indexes.^{[3][1]}
- Micro-partitions are mapped to worker nodes using consistent hashing and are intended to remain assigned to the same logical workers, maximizing cache reuse and limiting invalidation when the compute topology changes.^{[3][1]}

6.2 Semi-structured data and VARIANT

- Snowflake extends SQL with three notable types for semi-structured data: **VARIANT**, **ARRAY**, and **OBJECT**; VARIANT can hold scalar values as well as nested arrays and objects.^[2]
- Data can be loaded directly from JSON, Avro, or XML into VARIANT columns, enabling an **ELT** (Extract-Load-Transform) model: schemas can be inferred later, and transformations expressed in SQL, rather than baked into external ETL pipelines.^[2]
- Users can traverse nested data using functional SQL notation or JavaScript-like path expressions, flatten nested structures with lateral views, and re-nest using aggregates such as **ARRAY_AGG** and **OBJECT_AGG**.^[2]

6.3 Columnarization and optimistic conversion

- To retain both flexibility and performance, Snowflake performs **automatic type inference and column extraction** at the file (micro-partition) level: frequently occurring paths are extracted into separate typed, compressed columns stored alongside the serialized VARIANT payload.^[2]
- These derived columns participate in the same pruning machinery as native relational columns, enabling min/max pruning and statistics-driven optimization over semi-structured fields.^[2]

- Snowflake uses **optimistic conversion** for values like dates and numerics that may be encoded as strings; it stores both the converted typed value and (if necessary) the original string, so queries can benefit from typed operations and pruning without losing information (e.g., leading zeros in product codes).^[2]
- For path-existence pruning, Snowflake maintains Bloom filters over the set of paths present in each file, allowing the optimizer to skip files that provably do not contain the necessary fields, even if no detailed statistics are available.^[2]

7. Time travel, cloning, and data management features

7.1 Time travel and UNDROP

- Because Snowflake's MVCC is implemented via file sets, older versions of table files can be retained for a configurable retention period (up to ~90 days in the paper) and queried using **time travel**.^[2]
- SQL syntax supports selecting from a table **AT** a given timestamp or offset, or **BEFORE** a specific statement ID, and even joining different temporal versions of the same table within one query.^[2]
- Using the same metadata, Snowflake implements **UNDROP** for tables, schemas, and databases, allowing fast recovery from accidental DROP commands as long as files remain within the retention window.^[2]

7.2 Zero-copy cloning

- Snowflake supports **CLONE** operations on tables, schemas, and entire databases; cloning copies only metadata, so the new object initially references the same underlying files as the source.^[2]
- Subsequent modifications diverge the file sets, but the initial clone is effectively instantaneous and space-efficient, enabling workflows such as:
 - Pre-update snapshots.
 - Sandboxed analytical experiments.
 - Back-dated recovery via `CLONE ... BEFORE STATEMENT`.^[2]

8. Security and key management (overview)

- All user data (base tables, temporary files, network traffic) is encrypted end-to-end using AES-256 under a **hierarchical key model** rooted in AWS CloudHSM.^[2]

- The key hierarchy consists of root keys (in HSM), account keys, table keys, and derived file keys; each layer protects a narrower scope of data and keys are rotated and re-keyed according to NIST SP 800-57 life-cycle guidelines.^[2]
- File keys are derived from table keys and file names, so re-keying a table implies re-encrypting affected files; this is performed in the background on separate compute resources and coordinated atomically via metadata updates.^[2]

9. Lessons learned and impact

- The authors argue that, contrary to the “SQL on Hadoop will replace warehouses” narrative of the early 2010s, there remains strong demand for a **relational**, SQL-based warehouse that is cloud-native, elastic, and easy to operate.^[2]
- The multi-cluster, shared-data architecture and SaaS-first design enabled faster development, weekly online upgrades, and made performance largely a function of “how much compute the user is willing to pay for”, rather than manual tuning and physical design.^{[1][2]}
- Semi-structured data support turned out to be more important than initially expected: many customers had been using Hadoop essentially as a staging area to store and convert JSON before loading into RDBMSs; Snowflake’s VARIANT-based design allowed it to subsume part of that role.^[2]

10. BibTeX citations

```
@inproceedings{Dageville2016Snowflake,
  author    = {Benoit Dageville and Thierry Cruanes and Marcin {\.Z}ukowski and
              Vadim Antonov and Artin Avanes and Jon Bock and Jonathan Claybaugh and
              Daniel Engovatov and Martin Hentschel and Jiansheng Huang and
              Allison W. Lee and Ashish Motivala and Abdul Q. Munir and
              Steven Pelley and Peter Povinec and Greg Rahn and
              Spyridon Triantafyllis and Philipp Unterbrunner},
  title     = {The Snowflake Elastic Data Warehouse},
  booktitle = {Proceedings of the 2016 International Conference on Management of Data
              (SIGMOD)},
  year      = {2016},
  pages     = {215--226},
  publisher = {ACM},
  doi       = {10.1145/2882903.2903741}
}
```

```
@misc{Pavlo2024SnowflakeSlides,
```

```
author      = {Andy Pavlo},
title       = {Snowflake Database System},
howpublished = {Lecture 19, CMU 15-721 Advanced Database Systems, Spring 2024},
institution = {Carnegie Mellon University},
year        = {2024},
note        = {Course slides}
}
```

Note: I used the original Snowflake SIGMOD'16 paper and The class slides presentation pdfs

1. 19-snowflake pdf slides
2. Dageville-et-al.-The-Snowflake-Elastic-Data-Warehouse-SIGMOD16 pdf

The final content and any errors remain my responsibility